

CSCE 608 DATABASE SYSTEMS · MODULE 09 · LECTURE NOTES

Transactions and Concurrency Control

ACID, and what each letter actually costs.

MODULE OUTCOMES

State the ACID properties precisely.

Define conflict serializability and test for it.

Implement two-phase locking and explain why it works.

Explain deadlock detection and prevention.

Name the anomalies each isolation level permits.

1 ACID

Property	Guarantee	Mechanism
Atomicity	A transaction's effects are all applied or none are. A failure mid-transaction leaves no partial effect.	Logging and undo (Module 11).
Consistency	The database moves from one valid state to another.	Largely the application's responsibility. See below.
Isolation	Concurrent transactions produce the same result as if they had run one at a time.	Locking (this module) or multi-versioning (Module 10).
Durability	Once committed, the effects survive any subsequent crash.	Write-ahead logging and fsync (Module 11).

CONSISTENCY IS THE ODD ONE OUT

Atomicity, isolation, and durability are each implemented by a specific mechanism you can point to in the source: the log, the lock manager, the fsync call.

Consistency is not. The database enforces the constraints you have *declared* — primary keys, foreign keys, uniqueness, check constraints — and nothing beyond them.

'An account balance must never be negative' is enforced only if you declare it as a check constraint. Otherwise consistency means whatever invariants the application happens to maintain, and the database has no opinion.

Härder and Reuter coined the acronym in 1983, building on Gray's earlier work, and the C is widely regarded as having been included partly because the acronym required a vowel. This is not a criticism of the concept — consistency matters enormously — but of its presentation as a database-provided guarantee alongside three that genuinely are.

2 Serializability

THE CORRECTNESS CRITERION

A concurrent schedule is **serializable** if its effect is equivalent to executing the same transactions one after another, in *some* serial order.

Not necessarily the order in which they arrived. Any serial order counts, which is what makes the criterion achievable: we are not required to reproduce a particular outcome, only *an* outcome that could have arisen from running them one at a time.

Conflict serializability is the practically testable version. Two operations **conflict** if they are from different transactions, touch the same data item, and at least one is a write. Read–read does not conflict; read–write, write–read, and write–write do.

Build the conflict graph: a node per transaction, and an edge from T_1 to T_2 whenever T_1 has an operation that conflicts with and precedes one of T_2 's. **The schedule is conflict serializable if and only if that graph is acyclic**, and a topological sort of it gives the equivalent serial order.

Anomaly	Scenario
Dirty read	T_1 writes a row, T_2 reads it, T_1 rolls back. T_2 has read a value that never existed.
Non-repeatable read	T_1 reads a row, T_2 updates and commits, T_1 reads the same row again and sees different data.
Phantom	T_1 runs a range query, T_2 inserts a row matching the range, T_1 re-runs the query and sees an extra row. §3.
Lost update	T_1 and T_2 both read a value, both compute a new one from it, both write. One update is silently overwritten — the classic 'read-modify-write' bug.

Anomaly	Scenario
Write skew	Each transaction reads a set, checks an invariant, and writes a different row. Each write is individually valid; together they violate the invariant. The classic case: two doctors each check that at least one other doctor is on call, and both go off call. Snapshot isolation permits this (Module 10) and people do not expect it.

3 Two-phase locking

GROWING PHASE: may acquire locks, may NOT release any
 SHRINKING PHASE: may release locks, may NOT acquire any

Once a transaction releases its first lock, it may never acquire another. That one rule guarantees conflict serializability.

STRICT 2PL: hold ALL locks until commit or abort.
 Also prevents CASCADING ABORTS. <-- what real systems do

Lock modes: S (shared/read) X (exclusive/write)
 S/S compatible; everything else conflicts.

WHY STRICT 2PL RATHER THAN BASIC 2PL

Basic two-phase locking is sufficient for serializability: the growing/shrinking rule alone guarantees an acyclic conflict graph.

But it permits cascading aborts. If T_1 releases a lock after writing but before committing, T_2 can read that value. If T_1 then aborts, T_2 has read data that never existed and must abort too — along with anything that read from T_2 .

Strict 2PL holds every lock until commit or abort, so no transaction can ever read uncommitted data and the cascade is impossible. It also makes recovery simpler, because an abort never requires undoing other transactions.

The cost is holding locks slightly longer, which reduces concurrency. **Every real system accepts that trade**, because a cascading abort is a far worse problem than a marginally lower concurrency ceiling — it is unbounded in scope and arrives at unpredictable times.

Deadlock strategy	Mechanism	Cost
Detection	Maintain a waits-for graph and search it for cycles periodically. On finding one, abort a victim — usually the youngest transaction, or the one that has done least work.	The usual choice. Deadlocks are rare, so the periodic check is cheap and no transaction is aborted unnecessarily.
Prevention: wait-die	Each transaction carries a timestamp. If an <i>older</i> transaction needs a lock held by a younger one, it waits. If a <i>younger</i> one needs a lock held by an older, it aborts immediately.	Aborts transactions that would not actually have deadlocked , which is the price of never building the graph.

Deadlock strategy	Mechanism	Cost
Prevention: wound-wait	The reverse: an older transaction <i>aborts</i> the younger holder and takes the lock; a younger one waits.	Same cost. Favours older transactions, so long transactions make progress — which wait-die does not guarantee as strongly.
Timeout	Abort anything that has waited too long.	Crude. Aborts transactions that were merely slow, and tuning the threshold is guesswork.

PHANTOMS REQUIRE MORE THAN LOCKING THE ROWS THAT EXIST

T_1 executes `SELECT count(*) FROM people WHERE age > 30` and takes a shared lock on every row it found.

T_2 **inserts a new row with age 35**. That row was not locked — it did not exist when T_1 looked, so there was nothing to lock.

T_1 **re-runs its query and gets a different count**. The schedule is not serializable, and yet no lock was violated. **The problem is that the lock protects rows, and the query's semantics are about a *predicate*** — which covers rows that do not yet exist.

The fixes, in order of practicality: predicate locks, which lock the condition itself, are correct and impractical — testing whether two arbitrary predicates overlap is undecidable in general. **Index-range or next-key locks** lock the gaps between index entries, preventing insertion into a scanned range; this is approximate, practical, and what real systems implement. **Table locks** are correct, simple, and destroy concurrency.

4 Isolation levels

Level	Dirty read	Non-repeatable read	Phantom
Read uncommitted	Possible	Possible	Possible
Read committed	Prevented	Possible	Possible
Repeatable read	Prevented	Prevented	Possible
Serializable	Prevented	Prevented	Prevented

THREE THINGS ABOUT ISOLATION LEVELS THAT SURPRISE PEOPLE

The default is almost never serializable. PostgreSQL, Oracle, and SQL Server all default to Read Committed; MySQL's InnoDB defaults to Repeatable Read. **Most applications have never run under serializable isolation** and their developers are generally unaware of it.

The names do not reliably describe the behaviour. Oracle's SERIALIZABLE is in fact snapshot isolation, which is strictly weaker and permits write skew (Module 10). PostgreSQL's REPEATABLE READ is also snapshot isolation. The level name is a historical label, not a specification.

The standard's definition is by anomaly list, and the list is incomplete. Defining a level by which of three named anomalies it forbids says nothing about anomalies not on the list — and **write skew is not on the list**. Berenson, Bernstein, Gray, Melton, O'Neil and O'Neil's 'A Critique of ANSI SQL Isolation Levels' makes this case in detail and is worth reading.

So: determine empirically what your system provides, by writing the transactions that would exhibit each anomaly and observing what happens. The level name will not tell you, and Project 2 asks for exactly this demonstration.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
CMU 15-445 — Lectures 17–18, Transactions and Two-Phase Locking	ACID, serializability theory, 2PL, and deadlock handling.
Berenson et al. — A Critique of ANSI SQL Isolation Levels (free)	The §4 argument, from the people who built these systems. Short and important.
Gray & Reuter — Transaction Processing: Concepts and Techniques	The standard reference. Not free, and in most university libraries; Gray's individual papers are free and cover much of it.
Martin Kleppmann — Designing Data-Intensive Applications, chapter 7	The clearest modern explanation of isolation levels and the anomalies, with real incident examples.

Resource	What it covers for this module
Jepsen — consistency model analyses (free)	Empirical testing of what systems actually provide against what they claim. The practical counterpart to §4's advice.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write schedules exhibiting each of the five anomalies in §2 and reproduce each one in a real database at an appropriate isolation level.
- 2 Build conflict graphs for five schedules by hand and determine serializability. Verify your answers by reasoning about the outcomes.
- 3 Implement a lock manager with shared and exclusive modes and a wait queue.
- 4 Implement strict 2PL on top of it. Demonstrate that it prevents each of the anomalies.
- 5 Demonstrate a cascading abort under basic (non-strict) 2PL, then show strict 2PL preventing it.
- 6 Implement deadlock detection with a waits-for graph. Construct a deadlock and confirm detection and victim selection.
- 7 Implement wait-die and wound-wait. Measure the needless abort rate against detection on the same workload.
- 8 **Demonstrate a phantom:** run a range query twice with an insert in between, under repeatable read. Then enable serializable and show it prevented.
- 9 Determine your database's actual isolation behaviour by testing rather than reading the manual. Report any discrepancy with the level name.
- 10 Measure throughput against concurrency level under 2PL and identify where lock contention begins to dominate.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the four ACID properties with their mechanisms, and say which is the odd one out and why.
- 2 Define serializability and conflict serializability, and give the test.
- 3 Name five anomalies and describe each.
- 4 State the two-phase locking rule and say what it guarantees.
- 5 Why do real systems use strict 2PL rather than basic 2PL?
- 6 Compare deadlock detection with the two prevention schemes.

- 7 Why do row locks not prevent phantoms, and what does?
- 8 Name the four isolation levels and the anomalies each permits.
- 9 Give three reasons the isolation level names are misleading.